

HERMES A1

Sistema Autónomo de Adquisición y Telemetría

STM32F446RET6 — FreeRTOS — ADXL355 — Quectel EC25

Documentación Técnica v1.1.0

Antonio Avendaño Sanhueza

LIND Engineering

Mayo 2026 — Concepción, Chile

Índice

1. [Visión General](#) — sistema, hardware, evolución desde AWTAS
2. [Hardware](#) — pines, periféricos, NVIC, niveles eléctricos
3. [Arquitectura de Software](#) — tareas, colas, eventos, flujo de datos
4. [Gestión de Memoria](#) — mapa de memoria, Heap_4, dimensionamiento de cola
5. [Estado DMA y FPU](#) — configuración actual, impacto, recomendaciones
6. [Correcciones Aplicadas](#) — 8 bugs identificados y resueltos
7. [Análisis Comparativo](#) — 4 versiones del proyecto
8. [Evaluación UniKnect SDK](#) — viabilidad técnica
9. [Actividades Pendientes](#) — roadmap desde evaluación del prototipo
10. [Referencia API](#) — documentación Doxygen
11. [Compilación y Flash](#)
12. [Comandos CLI](#)
13. [Estructura del Proyecto](#)

Especificaciones Técnicas

MCU

STM32F446RET6

Cortex-M4 FPU, 512KB Flash, 128KB RAM

RTOS

FreeRTOS v10.3.1

CMSIS-RTOS v2 wrapper

FRECUENCIA

96 MHz

PLLCLK desde HSE

PACKAGE

LQFP64

64 pines

IDE

STM32CubeIDE

CubeMX v6.15.0 + GCC

HAL

STM32Cube FW_F4

V1.28.3

HEAP RTOS

40 KB

configTOTAL_HEAP_SIZE = 40960

TAREAS

5 (4 activas + 1 idle)

colas + eventos + mutex

1. Visión General

Firmware para STM32F446RET6 (Cortex-M4) del sistema **HERMES A1** — adquisición autónoma de vibraciones y telemetría vía 4G/LTE. Desarrollado por **Antonio Avendaño Sanhueza** para **LIND Engineering**, Concepción, Chile.

1.1 Componentes del Sistema

Componente	Modelo	Interfaz	Función
MCU	STM32F446RET6	—	Procesamiento, control, RTOS
Acelerómetro	ADXL355	SPI2 (8 MHz)	Medición de vibración 3 ejes, $\pm 2/4/8g$
Módem	Quectel EC25	USART1 (115200 baud)	Conectividad 4G/LTE, HTTP POST
Almacenamiento	microSD	SPI1 (8 MHz) + FatFs	Registro CSV de lecturas
Debug/CLI	—	USART2 (115200 baud)	Interfaz de comandos, diagnóstico

1.2 Principio de Operación

El sistema opera en modo **triggered acquisition**: el ADXL355 genera una interrupción (INT1) cuando la aceleración supera un umbral configurable. El firmware lee el FIFO del acelerómetro, almacena las lecturas en SD como archivos CSV, y las transmite a un servidor remoto vía HTTP/4G.

El pipeline de datos es **asíncrono y concurrente**: la adquisición del sensor, la escritura en SD, la transmisión por módem y la interfaz de usuario operan como tareas independientes de FreeRTOS, coordinadas mediante colas y event flags.

1.3 Evolución: AWTAS → HERMES A1

Dimensión	AWTAS (predecesor)	HERMES A1 (actual)
Arquitectura	Baremetal, super-loop en main.c (~58 KB)	FreeRTOS + CMSIS-RTOS v2, 5 tareas
Concurrencia	Secuencial: sensor → SD → módem (bloqueante)	Concurrente: cola + event flags + mutex
CLI	Menú de 1 carácter (m, l, r, o, t, i, q)	Comandos completos (help, status, accel, trigger, sdtest)
Sincronización	Variables volátiles + busy-wait	osMessageQueue, osEventFlags, osMutex
Tests	Ninguno	47 tests unitarios (Ceedling + Unity)

2. Hardware

2.1 Asignación de Periféricos

Periférico	Pines	Modo	Velocidad	Target	Nivel Lógico
SPI1	PA5, PA6, PA7, PB6(CS)	Full-Duplex Master	8.0 MB/s	microSD	3.3V
SPI2	PB13, PB14, PB15, PA4(CS)	Full-Duplex Master	8.0 MB/s	ADXL355	3.3V
USART1	PA9(TX), PA10(RX)	Asíncrono	115200 baud	Quectel EC25	1.8V (módem)
USART2	PA2(TX), PA3(RX)	Asíncrono	115200 baud	CLI / Debug	3.3V

✓ **Datos validados desde CubeMX .ioc**

Velocidades SPI y configuraciones validadas desde HERMES-A1-CMSIS.ioc (CubeMX v6.15.0). Frecuencia del sistema: 96 MHz (PLLCLK).

2.2 Pinout Completo

Pin	Etiqueta	Función	Configuración	Destino
SPI1 – microSD				
PA5	SPI1_SCK	Clock	AF5, Push-Pull	SD CLK
PA6	SPI1_MISO	Data In	AF5	SD MISO
PA7	SPI1_MOSI	Data Out	AF5	SD MOSI
PB6	SD_CS	Chip Select	Output PP, High, Speed HIGH	SD CS
SPI2 – ADXL355				
PA4	ADXL_CS	Chip Select	Output PP, High, Speed HIGH	ADXL355 CS
PB13	SPI2_SCK	Clock	AF5	ADXL355 SCLK
PB14	SPI2_MISO	Data In	AF5	ADXL355 MISO
PB15	SPI2_MOSI	Data Out	AF5	ADXL355 MOSI
USART1 – Quectel EC25				
PA9	USART1_TX	Transmit	AF7	EC25 RX
PA10	USART1_RX	Receive	AF7	EC25 TX
USART2 – CLI / Debug				
PA2	USART2_TX	Transmit	AF7	USB-serial RX
PA3	USART2_RX	Receive	AF7	USB-serial TX
GPIO – Control Módem				
PB0	HAT_PWR_OFF	Apagado EC25	Output PP, Low, Pull Down	EC25 PWR_OFF
PB1	MODEM_PWRKEY	Encendido EC25	Output PP, Low, Pull Down	EC25 PWRKEY
PB2	MODEM_RI	Ring Indicator	Input EXTI Falling, Pull Up	EC25 RI
GPIO – ADXL355				
PC0	ADXL_DRDY	Data Ready	Input	ADXL355 DRDY
PC7	ADXL_INT1	Interrupción	Input EXTI (prio 5), Pull Down	ADXL355 INT1
Depuración (SWD)				
PA13	SYS_JTMS-SWDIO	SWD Data	Alternate	ST-Link
PA14	SYS_JTCK-SWCLK	SWD Clock	Alternate	ST-Link
PB3	SYS_JTDO-SWO	SWO Trace	Alternate	ST-Link

2.3 Niveles Eléctricos y Tolerancias

Señal	Nivel MCU	Nivel Target	Notas
SPI1 (SD)	3.3V (VDD)	3.3V	Compatible directo. SD CS: Push-Pull, speed HIGH
SPI2 (ADXL355)	3.3V (VDD)	3.3V (VDD del ADXL355)	Compatible directo. ADXL_CS: Push-Pull, speed HIGH
USART1 (EC25)	3.3V (VDD)	1.8V (nivel módem)	⚠ Requiere level shifter o resistencias serie
USART2 (CLI)	3.3V (VDD)	3.3V (USB-serial)	Compatible directo
ADXL_INT1	3.3V (tolerante 5V)	3.3V	EXTI falling edge, pull-down interno
MODEM_RI	3.3V (tolerante 5V)	1.8V-2.8V (EC25)	EXTI falling edge, pull-up interno

⚠ ATENCIÓN

USART1 opera a 3.3V en el STM32 pero el EC25 espera 1.8V en sus pines UART. Verificar que el hardware incluye level shifting o resistencias serie de adaptación.

2.4 Configuración NVIC

IRQ	Fuente	Preempt Priority	Sub Priority	Propósito
EXTI9_5_IRQn	PC7 (ADXL_INT1)	5	0	Detección de movimiento (motion trigger)
TIM1_UP_TIM10_IRQn	TIM1	15	0	Base de tiempo del RTOS (SysTick sustituto)
PendSV_IRQn	FreeRTOS	15	0	Context switch del scheduler
SysTick_IRQn	SysTick	15	0	Timebase HAL

3. Arquitectura de Software

3.1 Tareas FreeRTOS

Tarea	Función	Prioridad CMSIS	Stack (words)	Stack (bytes)	Entrada
sensor_task	StartSensorTask	osPriorityHigh (40)	256	1024	EXTI (ADXL_INT1)
modem_task	StartModemTask	osPriorityAboveNormal (32)	512	2048	EVT_FILE_READY
file_task	StartFileTask	osPriorityNormal (24)	256	1024	sensor_queue
control_task	StartControlTask	osPriorityNormal (24)	1024	4096	UART2 (CLI)
defaultTask	StartDefaultTask	osPriorityNormal (24)	128	512	— (idle loop)

Nota sobre prioridades numéricas: CMSIS-RTOS v2 mapea prioridades sobre un rango de 56 niveles (configMAX_PRIORITIES). Los valores entre paréntesis son la prioridad numérica interna. Mayor número = mayor prioridad.

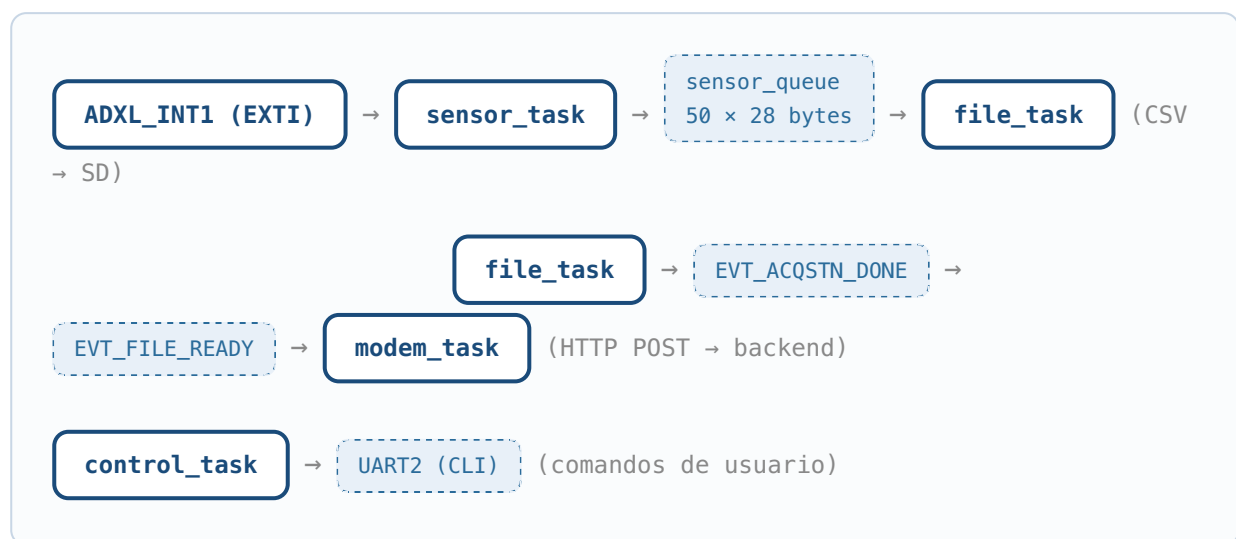
3.2 Objetos de Sincronización

Objeto	Tipo	Configuración	Uso
sensor_queue	osMessageQueue	50 slots × 28 bytes	sensor_task → file_task (lecturas)
sd_mutex	osMutex	Recursive	Protección de acceso a SD (FatFs)
sensor_event_flags	osEventFlags	5 flags	Coordinación entre tareas

3.3 Event Flags

Flag	Bit	Productor	Consumidor	Semántica
EVT_MOTION_DETECTED	(1<<0)	EXTI ISR / control_task	sensor_task	Iniciar adquisición
EVT_ACQSTN_DONE	(1<<1)	sensor_task	file_task	Adquisición completada
EVT_UPLOAD_DONE	(1<<2)	modem_task	—	Upload completado (monitor)
EVT_CFG_CHECK	(1<<3)	Timer periódico	control_task	Verificar configuración remota
EVT_FILE_READY	(1<<4)	file_task	modem_task	Archivo CSV listo para upload

3.4 Flujo de Datos



3.5 Tipo de Dato SensorReading_t

```
typedef struct {
    uint32_t timestamp_ms;    // 4 bytes – ms desde boot
    float    x_g;             // 4 bytes – aceleración X
    float    y_g;             // 4 bytes – aceleración Y
    float    z_g;             // 4 bytes – aceleración Z
    float    voltage;         // 4 bytes – reservado (siempre 0.0f)
    float    current;         // 4 bytes – reservado (siempre 0.0f)
    float    power;           // 4 bytes – reservado (siempre 0.0f)
} SensorReading_t;           // Total: 28 bytes
```

Nota: Los campos `voltage`, `current` y `power` están reservados para telemetría energética (pendiente §1.5 del roadmap). Actualmente siempre contienen `0.0f`.

3.6 Lógica de sensor_task

La tarea de sensor opera en un ciclo de estados implícito:

1. **Espera:** `osEventFlagsWait(EVT_MOTION_DETECTED, timeout=5000ms)` — heartbeat cada 5s
2. **Adquisición:** Lee ADXL355 cada 10ms (10 Hz), encola cada muestra en `sensor_queue`
3. **Settling detection:** Si la magnitud XY cae bajo `trigger_g` por 3s consecutivos ($\Delta < 0.005g$), termina la adquisición
4. **Earthquake rejection:** Si cualquier eje supera $\pm 2g$, aborta la adquisición (protección contra saturación)
5. **Notificación:** `EVT_ACQSTN_DONE` → `file_task` cierra archivo → `EVT_FILE_READY` → `modem_task` sube

Duración máxima de adquisición: 15 minutos (900,000 ms). Período de muestreo: 10ms (10 Hz) vía `osDelay(10)`.

4. Gestión de Memoria

4.1 Mapa de Memoria del STM32F446RE

Región	Dirección	Tamaño	Uso
Flash	0x0800 0000	512 KB	Código + constantes
SRAM	0x2000 0000	128 KB	Variables globales + heap + stacks
CCM RAM	0x1000 0000	64 KB	No utilizado (podría optimizarse)

4.2 Distribución del Heap FreeRTOS (Heap_4)

`configTOTAL_HEAP_SIZE = 40960` (40 KB) de los 128 KB de SRAM disponibles.

Componente	Tamaño	Cálculo
sensor_task stack	1024 B	256 words × 4 bytes
modem_task stack	2048 B	512 words × 4 bytes
file_task stack	1024 B	256 words × 4 bytes
control_task stack	4096 B	1024 words × 4 bytes
defaultTask stack	512 B	128 words × 4 bytes
sensor_queue	1400 B	50 slots × 28 bytes
Queue overhead	~200 B	Structura interna FreeRTOS
Mutex + EventFlags	~200 B	Structuras internas
Total asignado	~10,500 B	~26% del heap de 40 KB
Heap libre	~29,500 B	Disponible para malloc dinámico

4.3 Justificación de Heap_4

FreeRTOS ofrece 5 esquemas de heap. Se seleccionó **Heap_4** por las siguientes razones:

- **Coalescencia**: Heap_4 fusiona bloques libres adyacentes, reduciendo la fragmentación externa
- **Alloc + free**: A diferencia de Heap_1 (solo alloc) y Heap_2 (fragmentación alta), Heap_4 soporta liberación segura
- **First-fit**: Algoritmo determinista — siempre busca desde el inicio del heap

Riesgo residual: En operación continua (24/7), la fragmentación puede acumularse si hay patrones de alloc/free irregulares. Mitigación: usar `xPortGetFreeHeapSize()` periódicamente para monitorear.

4.4 Dimensionamiento de sensor_queue

La cola tiene **50 slots × 28 bytes = 1400 bytes**. El dimensionamiento se deriva de:

$$N_{\text{slots}} \geq (f_{\text{sample}} \times t_{\text{write_max}}) + \text{margen_de_seguridad}$$

Donde:

- `f_sample` = 10 Hz (1 muestra cada 10ms)
- `t_write_max` = tiempo máximo de escritura SD (FatFs `f_write` + `f_sync`)
- `margen_de_seguridad` = 2× para absorber picos de latencia

En el peor caso, si `file_task` tarda **500ms** en escribir un bloque a SD (latencia alta de FAT):

$$N_{\text{slots}} \geq (10 \text{ muestras/s} \times 0.5\text{s}) \times 2 = 10 \text{ slots} \rightarrow 50 \text{ slots proporciona } 5\times \text{ margen}$$

Con 50 slots, la cola puede absorber hasta **5 segundos** de latencia de escritura SD sin perder datos. A 10 Hz, esto equivale a 50 muestras = 5 segundos de adquisición bufferizada.

5. Estado DMA y FPU

5.1 FPU — Deshabilitada (Crítico)

Parámetro	Valor Actual	Recomendado
configENABLE_FPU	0 (deshabilitada)	1 (habilitada)

Impacto: sensor_task ejecuta operaciones de punto flotante en cada muestra:

- $\text{sqrtf}(x^2 + y^2)$ — magnitud para settling detection
- $\text{fabsf}(\text{current} - \text{prev})$ — delta para estabilidad
- 3 multiplicaciones float por muestra (x^2 , y^2 , división)

Sin FPU, cada operación float se emula por software (~50-200 ciclos por operación). A 10 Hz con 3-5 operaciones float por muestra, el overhead es de ~1,000-5,000 ciclos/s — manejable pero innecesario. El Cortex-M4 tiene FPU hardware; habilitarla es un cambio de 1 línea en FreeRTOSConfig.h.

⚠ CRÍTICO

Habilitar FPU (`configENABLE_FPU = 1`) antes de aumentar la tasa de muestreo o agregar procesamiento de señal (FFT, RMS, SNR). Sin FPU, el procesamiento de señal en tiempo real será inviable.

5.2 DMA — No Configurado

Interfaz	Método Actual	DMA Configurado	Impacto
SPI1 (SD)	Polling (<code>USE_DMA = 0</code>)	No	CPU bloqueada durante transferencias SPI
SPI2 (ADXL355)	Polling (HAL_SPI_Transmit/Receive)	No	CPU bloqueada durante lecturas FIFO
USART1 (EC25)	Polling (HAL_UART_Transmit/Receive)	No	CPU bloqueada durante comandos AT
USART2 (CLI)	Polling (printf → UART)	No	Menor impacto (solo debug)

Impacto en concurrencia: Sin DMA, las transferencias SPI y UART son bloqueantes. Durante una lectura de FIFO ADXL355 (32 muestras $\times$ 9 bytes = 288 bytes a 8 MHz $\approx$ 288 μ s), la CPU no puede atender otras tareas. Durante un comando AT al módem (puede tardar segundos), la tarea de módem bloquea el UART pero las demás tareas continúan (FreeRTOS es preventivo).

Recomendación: Implementar DMA para SPI2 (ADXL355) como prioridad P1. Esto libera la CPU durante lecturas burst del FIFO, permitiendo que otras tareas se ejecuten sin latencia adicional.

6. Correcciones de Código Aplicadas

#	Problema	Archivo	Solución
1	<code>strcasecmp()</code> no disponible en ARM GCC (newlib-nano)	main.c	Implementación local <code>strcasecmp_custom()</code>
2	Includes con mayúsculas incorrectas (<code>"Sd_spi.h"</code> vs <code>"sd_spi.h"</code>)	6 archivos	Corregido a minúsculas consistentes
3	Escrituras a registros EXTI sin deshabilitar interrupciones	control_task.c	<code>__disable_irq()</code> / <code>__enable_irq()</code> alrededor de operaciones EXTI
4	Falso positivo <code>EVT_ACQSTN_DONE</code> cuando <code>sdtest</code> abortaba	sensor_task.c	Guard <code>!sdbg_abort_acq</code> antes de <code>osEventFlagsSet</code>
5	Variable muerta <code>FRESULT fres</code> (warning de compilación)	main.c	Eliminada, llamada directa a <code>sd_mount()</code>
6	Credenciales vacías sin validación	credentials.h	Función <code>Credentials_Validate()</code> inline
7	FIFO ADXL355 leído con CS low/high por muestra (ineficiente)	adxl355.c	Burst read: CS bajo una vez, leer todas las muestras
8	Llamadas al módem antes de <code>osKernelStart()</code>	main.c	Comentadas con advertencia; mover a <code>modem_task</code>

7. Análisis Comparativo de Versiones

Se analizaron 4 versiones del proyecto para entender la evolución del código base:

Proyecto	Ubicación	Arquitectura	Estado
cmsis-test (local)	Directorio actual	FreeRTOS, 4 tareas + algo/, 47 tests	Versión de trabajo
cmsis-test (GitHub)	github.com/tonetooo/cmsis-test	FreeRTOS, 4 tareas, sin algo/	Semi-estable
cmsis (GitHub)	github.com/tonetooo/cmsis	FreeRTOS, 4 tareas, sin test_suite	Versión limpia
AWTAS	github.com/tonetooo/AWTAS	Baremetal, super-loop, state machines	Predecesor

7.1 Diferencias Clave

Feature	Local	cmsis-test GH	cmsis GH	AWTAS
Archivos .c / .h	16 / 19	14 / 17	13 / 16	10 / 10
Directorio algo/	✓	✗	✗	✗
Tests unitarios	✓ 47	✗	✗	✗
test_suite.c	✓	✓	✗	✗
RTOS	FreeRTOS 40KB	FreeRTOS	FreeRTOS	Baremetal
quectel_drive.c	866 líneas	~800	~800	744 líneas

La versión local es la más avanzada: incluye el directorio algo/ (lógica pura extraída para testing host), 47 tests unitarios Ceedling, y 8 correcciones de bugs aplicadas.

8. Evaluación de Quectel UniKnect SDK

Se evaluó el [Quectel UniKnect SDK](#) como reemplazo para la implementación manual de comandos AT en `quectel_drive.c` (866 líneas).

8.1 Compatibilidad

Componente	UniKnect SDK	Nuestro Proyecto	Veredicto
MCU	STM32F413RGT6 / VGT6	STM32F446RET6	✗ No compatible
Módem	EC200U, BG96	Quectel EC25	⚠ AT similar, no verificado
Build	CMake	CubeMX Makefile	✗ Requiere migración
RTOS	FreeRTOS bundled	FreeRTOS + CMSIS-RTOS v2	⚠ Posible conflicto

8.2 Veredicto

⚠ ATENCIÓN

No recomendado para la versión actual. El costo de portar al STM32F446RE + migrar a CMake + resolver conflictos de FreeRTOS supera los beneficios. Reconsiderar si se migra hardware a STM32F413 o se necesita MQTT/TCP/UDP.

9. Actividades Pendientes — Roadmap

Basado en la evaluación del prototipo HERMES A1 (Marzo 2026), se identifican las siguientes actividades para el sistema de adquisición de datos. Prioridades definidas según impacto en operación 24/7 en terreno.

9.1 Prioridad 1 — Crítico (operación confiable en terreno)

#	Actividad	Implementación	Archivos
9.1.1	DMA para adquisición de datos DMA en SPI2 (ADXL355) para lecturas burst del FIFO. Actualmente polling bloquea la CPU durante cada transferencia. Con DMA, la CPU queda libre para atender otras tareas durante la lectura de hasta 32 muestras (288 bytes).	Configurar DMA2 Stream0 o Stream3 en modo circular/normal, trigger desde ADXL355 FIFO watermark	adxl355.c, stm32f4xx_hal_msp.c, CubeMX .ioc
9.1.2	Habilitar FPU Cambiar configENABLE_FPU de 0 a 1. Necesario para cualquier procesamiento de señal en tiempo real (FFT, RMS, SNR). Sin FPU, cada operación float se emula por software (~100x más lento).	1 línea en FreeRTOSConfig.h	FreeRTOSConfig.h
9.1.3	Adquisición en tiempo continuo Modo de operación permanente sin depender exclusivamente de trigger por evento. Útil para caracterización de línea base estructural y monitoreo continuo.	Agregar modo CONTINUOUS al estado de sensor_task, configurable vía CLI	sensor_task.c, control_task.c
9.1.4	Calibración del acelerómetro LevelToZero ya existe (512 muestras en reposo, calcula offsets XYZ). Falta: calibración de sensibilidad efectiva, linealidad, y ruido propio del sensor mediante comparación con referencia.	Comando CLI calibrate que ejecute rutina de calibración y guarde coeficientes en flash	adxl355.c, control_task.c
9.1.5	Watchdog Implementar watchdog de hardware (IWDG del STM32) y/o software para detectar bloqueos del firmware y ejecutar reinicios automáticos.	CubeMX IWDG + HAL_IWDG_Refresh() en cada tarea periódica	CubeMX .ioc, freertos.c
9.1.6	Buffer local ante pérdida de conectividad Datos se guardan en SD y se suben vía HTTP. Si el módem falla, no hay	Cola de archivos pendientes en file_task,	file_task.c, modem_task.c

	mecanismo de reintento automático ni cola de archivos pendientes.	reintento periódico en modem_task	
9.1.7	Sincronización horaria Timestamps actuales son osKernelGetTickCount() (ms desde boot). Se necesita hora real para correlacionar datos con eventos externos.	NTP vía módem (AT+QNTP) o sincronización por red celular al iniciar conexión	quectel_drive.c, sensor_task.c
9.1.8	Optimización de consumo energético Desactivar LEDs indicadores, usar modos de bajo consumo del STM32 (Sleep/Stop), optimizar ciclos de adquisición y transmisión.	HAL_SuspendTick() en idle, apagar periféricos no usados, reducir ODR del ADXL355 en reposo	sensor_task.c, freertos.c, CubeMX .ioc

9.2 Prioridad 2 — Alto (confiabilidad y diagnóstico remoto)

#	Actividad	Implementación	Archivos
9.2.1	Métricas de calidad de señal SNR, ruido de fondo, saturación, RSSI, tipo de red	AT+CSQ para RSSI, cálculo RMS en sensor_task	sensor_task.c, quectel_drive.c
9.2.2	Telemetría energética integrada Campos voltage/current/power en SensorReading_t	Configurar ADC1 para medir voltaje batería y corriente	sensor_task.c, stm32f4xx_hal_msp.c
9.2.3	Gestión remota de configuración Descargar y actualizar config sin reiniciar	DownloadConfig() ya existe; agregar parser + recarga runtime	quectel_drive.c
9.2.4	Sistema de alarmas operativas Batería crítica, vibración fuera de rango, manipulación	HTTP POST de alerta a endpoint dedicado	sensor_task.c, modem_task.c
9.2.5	Monitoreo del estado interno Heap libre, uptime, errores, reinicios	Comando CLI diag + registro periódico en CSV separado	control_task.c
9.2.6			adxl355.c, sensor_algo.c

	Compensación térmica Drift térmico del ADXL355	Leer registro TEMPERATURE del ADXL355, aplicar corrección	
9.2.7	Caracterización del enlace celular RSSI, BER, tipo de red en distintas condiciones	Evaluar con AT commands, diseñar red de adaptación de antena	quectel_drive.c

9.3 Prioridad 3 — Evolución (escalabilidad)

#	Actividad	Implementación	Archivos
9.3.1	Arquitectura multisensor I²C Habilitar ADXL355 en modo I ² C, integrar multiplexor TCA9548A, soportar múltiples sensores distribuidos hasta ~30m. Permite red de nodos de vibración en estructuras grandes.	Configurar I2C1/I2C2 en CubeMX, implementar driver TCA9548A, refactorizar adxl355.c para soportar múltiples instancias	adxl355.c, CubeMX .ioc, nuevo tca9548a.c
9.3.2	Reinicio remoto Mecanismo de reinicio desde la plataforma de monitoreo. Podría implementarse vía comando HTTP especial o GPIO watchdog. Permite recuperación sin visita a terreno.	Endpoint HTTP dedicado en backend → modem_task parsea comando → NVIC_SystemReset(). Alternativa: GPIO conectado a NRST del STM32.	quectel_drive.c, modem_task.c
9.3.3	Migración del protocolo de transmisión Evaluar FTP/FTPS como reemplazo o complemento de HTTP POST para transferencia de archivos robusta 24/7. FTP permite resume de transferencias interrumpidas, mejor para operación continua.	Implementar cliente FTP en quectel_drive.c usando comandos AT del EC25 (AT+QFTPCMD). Evaluar FTPS para cifrado.	quectel_drive.c

9.4 Matriz de Prioridades

#	Actividad	Prioridad	Esfuerzo	Dependencias
1	DMA para SPI2	P1	Medio	CubeMX reconfig
2	Habilitar FPU	P1	Bajo (1 línea)	—
3	Adquisición continua	P1	Medio	Modo de estado en sensor_task
4	Calibración acelerómetro	P1	Alto	Requiere hardware de referencia
5	Watchdog	P1	Bajo	CubeMX IWDG
6	Buffer conectividad	P1	Medio	Cola de archivos en file_task
7	Sincronización horaria	P1	Medio	AT+QNTTP o NTP
8	Optimización consumo	P1	Alto	Múltiples subsistemas
9	Métricas calidad señal	P2	Medio	AT+CSQ, RMS calculation
10	Telemetría energética	P2	Medio	ADC1 + DMA
11	Gestión remota config	P2	Medio	DownloadConfig existente
12	Alarmas operativas	P2	Bajo	HTTP POST endpoint
13	Monitoreo interno	P2	Bajo	FreeRTOS API
14	Compensación térmica	P2	Medio	ADXL355 TEMPERATURE register
15	Caracterización celular	P2	Alto	Pruebas de campo
16	Multisensor I ² C	P3	Alto	TCA9548A, I2C bus
17	Reinicio remoto	P3	Bajo	HTTP command o GPIO
18	Migración FTP/FTPS	P3	Alto	Nuevo protocolo

10. Referencia API

La referencia completa de la API está generada automáticamente por Doxygen:

Salida Doxygen HTML: `docs/doxygen/html/index.html`

Abrir en navegador para documentación interactiva con grafos de llamadas, jerarquía de funciones y navegación de código fuente.

Módulos Principales

Módulo	Archivos	Responsabilidad
Driver ADXL355	<code>adxl355.c/h</code>	SPI2, lectura FIFO, conversión raw → g, LevelToZero
Driver Quectel EC25	<code>quectel_drive.c/h</code>	PowerOn, BringUpNetwork, SendAT, UploadFile, DownloadConfig
Driver SD	<code>sd_spi.c/h</code>	SPI1, bloques 512B, FatFs diskio
Tareas RTOS	<code>tasks/*.c</code>	sensor, modem, file, control — pipeline de datos
Algoritmos	<code>algo/*.c/h</code>	Lógica pura: motion detection, settling, CLI parser, CSV format
CLI	<code>control_task.c</code>	Parser de comandos, ejecución, output por UART2
Tests	<code>test_suite.c/h</code>	Diagnóstico: SD test, modem test, sensor test

12. Comandos CLI

Comando	Descripción
help	Lista de comandos disponibles
status	Estado del sistema y umbral actual
accel	Lectura actual del acelerómetro
trigger	Umbral de disparo actual en G
trigger <valor>	Establecer umbral (0–10 G)
log	Lista archivos CSV en SD
test	Simula evento de movimiento (prueba de pipeline)
sdtest	Adquisición forzada 10s + tabla ASCII en SD
modem_on	Enciende módem + prueba sincronización AT

11. Compilación y Flash

Compilar

```
# STM32CubeIDE: importar proyecto y compilar
# Línea de comandos (arm-none-eabi-gcc en PATH):
make -j4
```

Flashear

```
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg \
  -c "program build/HERMES-A1-CMSIS.elf verify reset exit"
```

Conexión CLI

```
picocom -b 115200 /dev/tty.usbmodem*
# 0
screen /dev/tty.usbmodem* 115200
```

Tests Unitarios

```
cd test && ceedling test:all
# 47 tests: sensor_algo (23) + cli_algo (18) + csv_algo (6)
```

13. Estructura del Proyecto

Ruta	Contenido
Core/Inc/	Headers: <code>main.h</code> , <code>adxl355.h</code> , <code>quectel_drive.h</code> , <code>sd_spi.h</code> , <code>tasks.h</code> , <code>test_suite.h</code> , <code>credentials.h</code>
Core/Inc/algo/	Lógica pura (test host): <code>sensor_algo.h</code> , <code>cli_algo.h</code> , <code>csv_algo.h</code>
Core/Src/	Implementación: <code>main.c</code> , <code>adxl355.c</code> , <code>quectel_drive.c</code> , <code>sd_spi.c</code> , <code>modem_sim.c</code> , <code>stm32f4xx_hal_msp.c</code> , <code>stm32f4xx_it.c</code> , <code>test_suite.c</code>
Core/Src/algo/	Lógica pura: <code>sensor_algo.c</code> , <code>cli_algo.c</code> , <code>csv_algo.c</code>
Core/Src/tasks/	Tareas RTOS: <code>sensor_task.c</code> , <code>file_task.c</code> , <code>modem_task.c</code> , <code>control_task.c</code>
Core/Startup/	Startup assembly + linker script
Drivers/	STM32 HAL + CMSIS
FATFS/	FatFs middleware
Middlewares/	FreeRTOS + FatFs
Lib/FreeRTOS-LTS/	FreeRTOS LTS submodule
test/	Ceedling + Unity: 47 tests (<code>project.yml</code> + <code>test/</code>)
docs/es/	Documentación español: HTML + PDF
docs/doxygen/	API reference generada por Doxygen
Doxyfile	Configuración Doxygen
Makefile	Build system (arm-none-eabi-gcc)
HERMES-A1-CMSIS.ioc	STM32CubeMX project config
STM32F446RETX_FLASH.ld	Linker script